

HOJA DE AYUDA: CONCEPTOS BÁSICOS

Ten este documento a la mano mientras realizas los ejercicios.

1. ¿Por qué MySQL y no seguir en memoria RAM?

Característica	Memoria RAM (Semana 7-8)	MySQL (Semana 9)
Persistencia	Se pierde al reiniciar	Permanece entre sesiones
Concurrencia	Solo un usuario	Múltiples usuarios simultáneos
Volumen	Limitada por memoria del servidor	Maneja millones de registros
Integridad	Sin validaciones de estructura	Restricciones (FK, UNIQUE, CHECK)
Consultas complejas	Arreglos en JavaScript	SQL con JOIN, GROUP BY, SUM

2. Tipos de datos MySQL más usados en este proyecto

Tipo	Uso	Ejemplo
INT AUTO_INCREMENT	Llaves primarias	<code>id</code>
<code>VARCHAR(n)</code>	Textos cortos	<code>nombre VARCHAR(100)</code>
<code>DECIMAL(10,2)</code>	Montos de dinero	<code>monto DECIMAL(10,2) → 99999999.99</code>
<code>ENUM('A', 'B')</code>	Valores fijos	<code>tipo ENUM('Ingreso', 'Egreso')</code>
<code>DATE</code>	Solo fecha	<code>fecha DATE → '2026-10-01'</code>
<code>TIMESTAMP</code>	Fecha y hora automática	<code>fecha_creacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP</code>

3. Foreign Keys y cardinalidad

Foreign Key (FK): Campo que referencia la PK de otra tabla. Garantiza integridad referencial.

```
FOREIGN KEY (contacto_id) REFERENCES contactos(id) ON DELETE SET NULL
```

- `contacto_id` en `movimientos` apunta a `id` en `contactos`
- `ON DELETE SET NULL`: si se elimina un contacto, sus movimientos no se borran, solo queda `contacto_id = NULL`

Cardinalidad 1:N: Un contacto puede tener **muchos** movimientos, pero un movimiento pertenece a **un solo** contacto.

4. Códigos de estado HTTP en este proyecto

Código	Nombre	Cuándo usarlo en el ERP
200	OK	GET y PUT exitosos
201	Created	POST exitoso (recurso creado)
400	Bad Request	Validación fallida (campos vacíos, monto negativo, RFC duplicado)
404	Not Found	ID no existe en la BD
500	Server Error	Error de conexión a MySQL, bug en el código

5. Diferencia entre `db.query()` con arreglos en memoria

Antes (memoria RAM):

```
let contactos = [];
app.get('/api/contactos', (req, res) => {
  res.json(contactos); // Devuelve el arreglo en memoria
});
```

Ahora (MySQL):

```
app.get('/api/contactos', async (req, res) => {
  const [rows] = await db.query('SELECT * FROM contactos');
  res.json(rows); // Devuelve resultado de la consulta SQL
});
```

6. Errores comunes y soluciones

Error	Causa	Solución
<code>ER_ACCESS_DENIED_ERROR</code>	Credenciales incorrectas en <code>db.js</code>	Verifica usuario, password y permisos
<code>ER_DUP_ENTRY</code>	RFC duplicado	Usa un RFC único o elimina el registro duplicado
<code>ECONNREFUSED</code>	MySQL no está corriendo	Inicia el servicio MySQL
Los datos no persisten	El servidor aún usa arreglos	Verifica que uses <code>db.query()</code> en todos los endpoints
Cannot POST /api/...	Falta <code>express.json()</code>	Agrega <code>app.use(express.json())</code> antes de las rutas

7. Comandos MySQL útiles

```
SHOW DATABASES;      -- Ver bases de datos
USE erp_contable_jdc; -- Seleccionar BD (reemplaza con tu BD)
SHOW TABLES;        -- Ver tablas
```

```
DESCRIBE contactos;           -- Ver estructura de tabla
SELECT * FROM contactos;      -- Ver todos los registros
ALTER TABLE contactos ADD telefono VARCHAR(20); -- Agregar columna (Reto Prueba de Fuego #2)
DELETE FROM contactos WHERE id=1; -- Eliminar registro específico
DROP DATABASE erp_contable_jdc; -- ⚠ Eliminar BD completa (cuidado)
```

8. Tips para la Prueba de Fuego

- **Respira:** 3 minutos es tiempo suficiente si sabes qué buscar.
 - **Lee el error:** La consola de Node.js te dice exactamente en qué línea falló.
 - **Usa internet:** Si olvidas la sintaxis de `ALTER TABLE`, búscala. Lo importante es saber *qué* buscar.
 - **Prueba en Postman:** Después de cada modificación, verifica en Postman que el endpoint funcione.
 - **No entres en pánico con el Reto 3:** Cambiar la contraseña en `db.js` es solo editar un string. No hay que reiniciar MySQL ni hacer nada complejo.
-